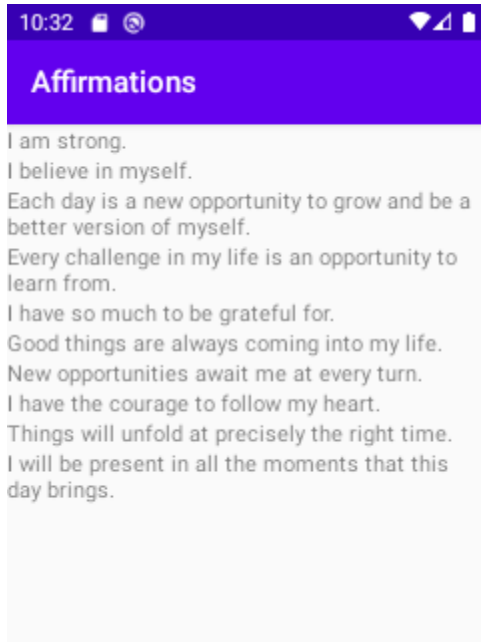


Lab 5.5. Display a list of images using cards

1. Before you begin

In the previous codelab, you created an [Affirmations app](#) that displays a list of text in a RecyclerView.



In this follow-up codelab, you add an inspiring image to each affirmation of your app. You will display the text and image for each affirmation within a card, using the `MaterialCardView` widget from the Material Components for Android library. Then you will finish the app by polishing the UI to create a more cohesive and beautiful user experience. This is a screenshot of the completed app:

1:46



Affirmations



I am strong.



I believe in myself.



Prerequisites

- Can add image resources to an app.
- Comfortable modifying an XML layout.
- Able to create an app that displays a list of text in a `RecyclerView`.
- Able to create an adapter for a `RecyclerView`.

What you'll learn

- How to add images to the list of displayed affirmations in a `RecyclerView`.
- How to use `MaterialCardView` in a `RecyclerView` item layout.
- How to make visual changes in the UI to make the app look more polished.

What you'll build

- A polished Affirmations app that uses a `RecyclerView` to display a list of cards. Each card contains an image and affirmation text.

What you need

- A computer with Android Studio version 4.1 or higher installed.
- Access to an internet connection to download image files.
- The Affirmations app from the previous **Create an Affirmations app** codelab. (No starter code is provided. Creating the app is a prerequisite.)

2. Adding images to the list items

So far you have created an adapter `ItemAdapter` to display affirmation strings in a `RecyclerView`. Functionally this works great, but visually it is not very appealing. In this task, you will modify the list item layout and adapter code to display images with the affirmations.

Download the images

1. To start, open up the Affirmations app project in Android Studio from the previous codelab. If you don't have this project, go through the steps in the previous codelab to create that project. Then return here.
2. Next [download the image files](#) onto your computer. There should be ten images, one for each affirmation in your app. The files should be named from `image1.jpg` to `image10.jpg`.
3. Copy the images from your computer into your project's **res > drawable** folder (`app/src/main/res/drawable`) within Android Studio. Once these resources have been added to your app, you will be able to access these images from your code using their

resource IDs, such as `R.drawable.image1`. (You may have to rebuild your code for Android Studio to find the image.)

Now the images are ready to use in the app.

Add support for images in the Affirmation class

In this step, you'll add a property in the `Affirmation` data class to hold a value for an image resource ID. That way a single `Affirmation` object instance will contain a resource ID for the text of the affirmation and a resource ID for the image of the affirmation.

1. Open the `Affirmation.kt` file within the `model` package.
2. Modify the constructor of the `Affirmation` class by adding another `Int` parameter named `imageResourceId`.

Using resource annotations

Both `stringResourceId` and `imageResourceId` are integer values. Although this looks okay, the caller could accidentally pass in the arguments in the wrong order (`imageResourceId` first instead of `stringResourceId`).

To avoid this, you can use `Resource` annotations. Annotations are useful because they add additional info to classes, methods, or parameters. Annotations are always declared with an `@` symbol. In this case, add the `@StringRes` annotation to your string resource ID property and `@DrawableRes` annotation to your drawable resource ID property. Then you will get a warning if you supply the wrong type of resource ID.

1. Add the `@StringRes` annotation to `stringResourceId`.
2. Add the `@DrawableRes` annotation to `imageResourceId`.
3. Make sure the imports `androidx.annotation.DrawableRes` and `androidx.annotation.StringRes` are added at the top of your file after the package declaration.

`Affirmation.kt`

```
package com.example.affirmations.model

import androidx.annotation.DrawableRes
import androidx.annotation.StringRes

data class Affirmation(
    @StringRes val stringResourceId: Int,
    @DrawableRes val imageResourceId: Int
)
```

Initialize list of affirmations with images

Now that you've changed the constructor of the `Affirmation` class, you need to update the `Datasource` class. Pass in an image resource ID to each `Affirmation` object that is initialized.

1. Open `Datasource.kt`. You should see an error for each instantiation of `Affirmation`.
2. For each `Affirmation`, add the resource ID of an image as an argument, such as `R.drawable.image1`.

`Datasource.kt`

```
package com.example.affirmations.data

import com.example.affirmations.R
import com.example.affirmations.model.Affirmation

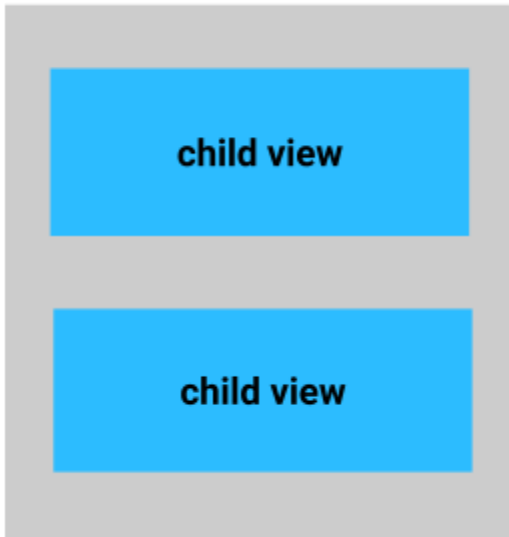
class Datasource() {

    fun loadAffirmations(): List<Affirmation> {
        return listOf<Affirmation>(
            Affirmation(R.string.affirmation1, R.drawable.image1),
            Affirmation(R.string.affirmation2, R.drawable.image2),
            Affirmation(R.string.affirmation3, R.drawable.image3),
            Affirmation(R.string.affirmation4, R.drawable.image4),
            Affirmation(R.string.affirmation5, R.drawable.image5),
            Affirmation(R.string.affirmation6, R.drawable.image6),
            Affirmation(R.string.affirmation7, R.drawable.image7),
            Affirmation(R.string.affirmation8, R.drawable.image8),
            Affirmation(R.string.affirmation9, R.drawable.image9),
            Affirmation(R.string.affirmation10, R.drawable.image10)
        )
    }
}
```

Add an `ImageView` to the list item layout

To show an image for each affirmation in your list, you need to add an `ImageView` to your item layout. Because you now have two views (a `TextView` and `ImageView`), you need to place them as children views within a `ViewGroup`. To arrange the views in a vertical column, you can use a `LinearLayout`. `LinearLayout` aligns all child views in a single direction, vertically or horizontally.

LinearLayout



1. Open **res > layout > list_item.xml**. Add a `LinearLayout` around the existing `TextView` and set the `orientation` property to `vertical`.
2. Move the `xmlns` schema declaration line from the `TextView` element to the `LinearLayout` element to get rid of the error.

list_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:id="@+id/item_title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

</LinearLayout>
```

3. Inside the `LinearLayout`, before the `TextView`, add an `ImageView` with a resource ID of `item_image`.
4. Set the `ImageView`'s width to `match_parent` and height to `194dp`. Depending on screen size, this value should show a few cards on screen at any given time.
5. Set the `scaleType` to `centerCrop`.
6. Set the `importantForAccessibility` attribute to `no` since the image is used for decorative purposes.

```
<ImageView
    android:layout_width="match_parent"
```

```
android:layout_height="194dp"
android:id="@+id/item_image"
android:importantForAccessibility="no"
android:scaleType="centerCrop" />
```

Update the ItemAdapter to set the image

1. Open `adapter/ItemAdapter.kt` (**app > java > adapter > ItemAdapter**)
2. Go to the `ItemViewHolder` class.
3. An `ItemViewHolder` instance should hold a reference to the `TextView` and a reference to the `ImageView` in the list item layout. Make the following change.

Below the initialization of the `textView` property, add a `val` called `imageView`. Use `findViewById()` to find the reference to the `ImageView` with ID `item_image` and assign it to the `imageView` property.

`ItemAdapter.kt`

```
class ItemViewHolder(private val view: View): RecyclerView.ViewHolder(view) {
    val textView: TextView = view.findViewById(R.id.item_title)
    val imageView: ImageView = view.findViewById(R.id.item_image)
}
```

4. Find the `onBindViewHolder()` function in `ItemAdapter`.
5. Previously you set the affirmation's `stringResourceId` on to `textView` in the `ItemViewHolder`. Now set the affirmation item's `imageResourceId` onto the `ImageView` of the list item view.

```
override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
    val item = dataset[position]
    holder.textView.text =
context.resources.getString(item.stringResourceId)
    holder.imageView.setImageResource(item.imageResourceId)
}
```

6. Run the app and scroll through the list of affirmations.

2:03



Affirmations



I am strong.



I believe in myself.



Each day is a new opportunity to grow and be a better version of myself.



The app looks much prettier with images! Yet you can still improve the app UI. In the next section you will make small adjustments to the app to improve the UI.

3. Polishing the UI

So far you've built a functional app that consists of a list of affirmation strings and images. In this section, you'll see how small changes in the code and XML can make the app look more polished.

Add padding

To start with, add some whitespace between the items in the list.

Tip: You can make layout changes in the XML as shown here, or you can make them in the **Attributes** panel in **Design** view, whichever you prefer.

1. Open `item_list.xml` (**app > res > layout > item_list.xml**) and add 16dp padding to the existing `LinearLayout`.

`list_item.xml`

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">
```

2. Add 16dp padding to the `item_title` `TextView`.
3. In the `TextView`, set the `textAppearance` attribute to `?attr/textAppearanceHeadline6`. [textAppearance](#) is an attribute that allows you to define text-specific styling. For other possible predefined text appearance values, you can see the [TextAppearances](#) section in this [blogpost on Common Theme Attributes](#).

```
<TextView
    android:id="@+id/item_title"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:padding="16dp"
    android:textAppearance="?attr/textAppearanceHeadline6" />
```

4. Run the app. Do you think the list looks better?

1:59



Affirmations



I am strong.



I believe in myself.



Use cards

It is still hard to tell if an image belongs to the affirmation text above or below that image. To fix this you can use a **Card** view. A Card view provides an easy way to contain a group of views while providing a consistent style for the container. For more Material Design guidance on using cards, check out this [guide on cards](#).

1. Add a `MaterialCardView` around the existing `LinearLayout`.
2. Once again, move the schema declaration from `LinearLayout` into `MaterialCardView`.
3. Set the `layout_width` of the `MaterialCardView` to `match_parent`, and the `layout_height` to `wrap_content`.
4. Add a `layout_margin` of `8dp`.
5. Remove the padding in the `LinearLayout`, so you don't have too much whitespace.
6. Now run the app again. Can you tell each affirmation apart better with `MaterialCardView`?

`list_item.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <ImageView
            android:id="@+id/item_image"
            android:layout_width="match_parent"
            android:layout_height="194dp"
            android:importantForAccessibility="no"
            android:scaleType="centerCrop" />

        <TextView
            android:id="@+id/item_title"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:textAppearance="?attr/textAppearanceHeadline6" />

    </LinearLayout>

</com.google.android.material.card.MaterialCardView>
```

1:56



Affirmations



I am strong.



I believe in myself.



Change the app theme colors

The default app theme color may not be as calming as some other choices you could make. In this task, you will change the app theme color to blue. You can then change it again using your own ideas!

You can find pre-defined shades of blue from the Material Design color palette from this [link](#).

In this codelab, you will be using the following colors from the Material Design palette:

- blue_200: #FF90CAF9
- blue_500: #FF2196F3
- blue_700: #FF1976D2

Add color resources

Define colors used within your app in a centralized place: the `colors.xml` file.

1. Open `colors.xml` (**res > values > colors.xml**).
2. Add new color resources to the file for the blue colors defined below:

```
<color name="blue_200">#FF90CAF9</color>
<color name="blue_500">#FF2196F3</color>
<color name="blue_700">#FF1976D2</color>
```

Change the theme colors

Now that you have new color resources, you can use them in your theme.

1. Open `themes.xml` (**res > values > themes > themes.xml**).
2. Find the `<!-- Primary brand color. -->` section.
3. Add or change `colorPrimary` to use `@color/blue_500`.
4. Add or change `colorPrimaryVariant` to use `@color/blue_700`.

```
<item name="colorPrimary">@color/blue_500</item>
<item name="colorPrimaryVariant">@color/blue_700</item>
```

5. Run the app. You should see the app bar color is changed to blue.

1:46



Affirmations



I am strong.



I believe in myself.



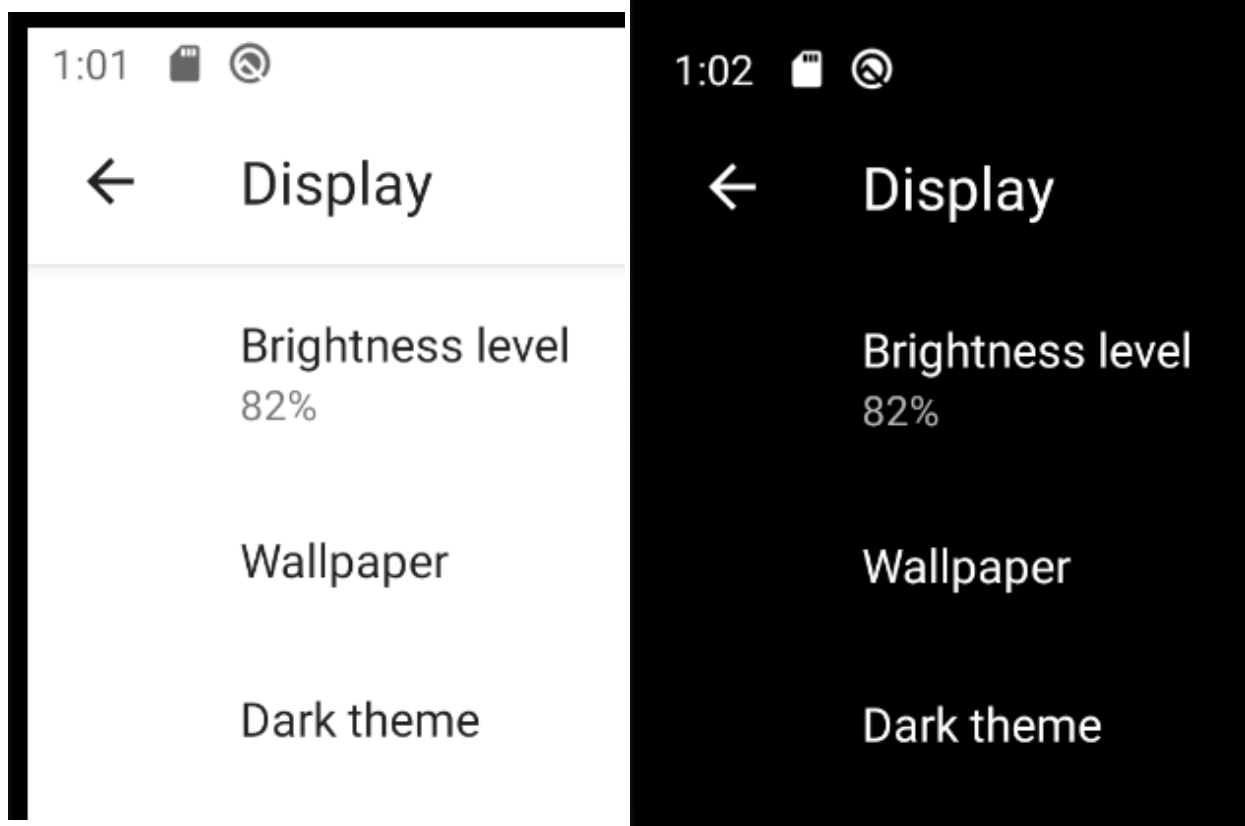
Update the dark theme colors

It's good to choose more desaturated colors for the [dark theme](#) of the app.

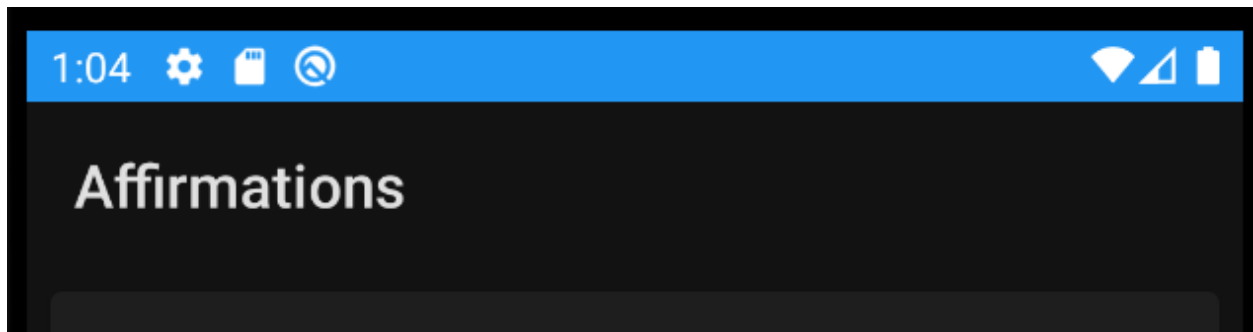
1. Open the dark theme `themes.xml` file (**themes > themes.xml (night)**).
2. Add or change the `colorPrimary` and `colorPrimaryVariant` theme attributes as follows:

```
<item name="colorPrimary">@color/blue_200</item>
<item name="colorPrimaryVariant">@color/blue_500</item>
```

3. Run your app.
4. In the **Settings** of your device, turn on the **Dark Theme**.



5. Your app switches to the **Dark Theme**. Verify that it looks like the screenshot below:



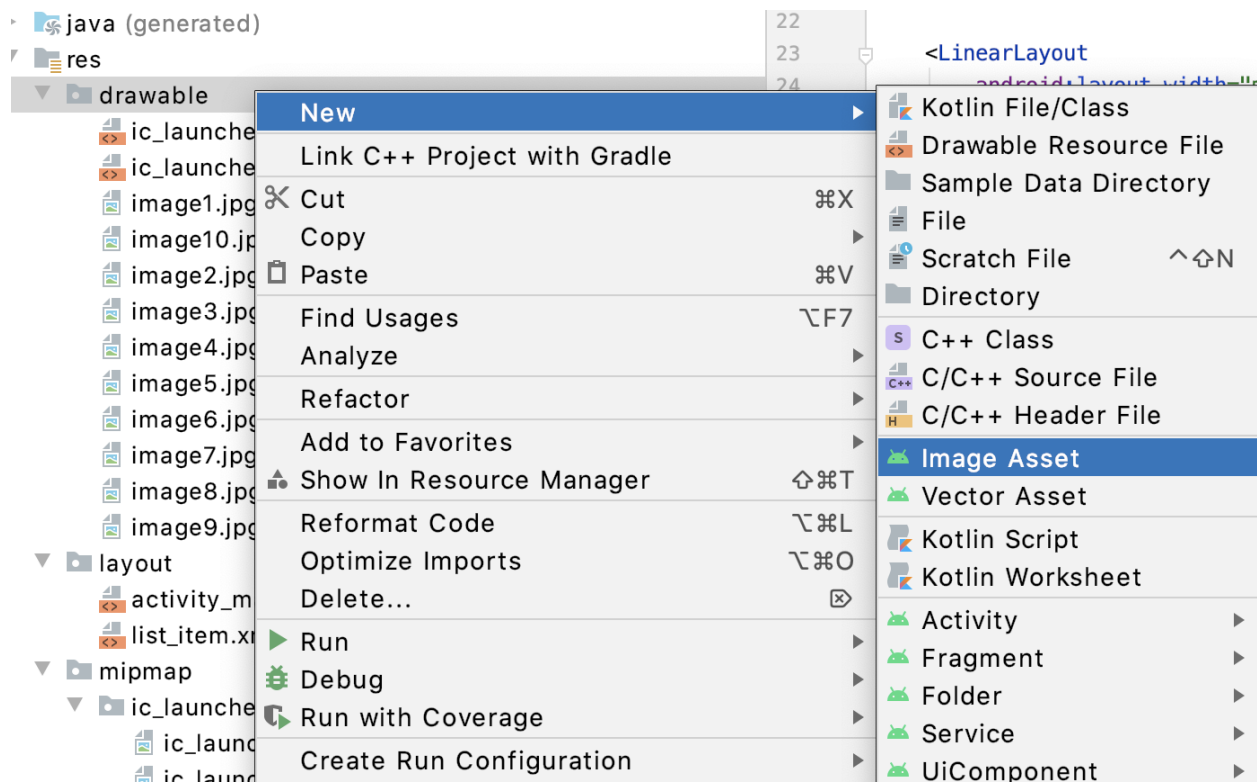
Note: app bar color. You may be wondering why your specified primary color for the dark theme is not showing in the app bar. The dark app bar is by design. In dark theme, the app bar and other large areas are by default shown with a dark background (`colorSurface`) instead of the primary color. This is because Material dark theme recommends less use of bright colors on large surfaces. Buttons or other small accents will show the defined primary color.

6. At this point, you can also remove unused colors in your `colors.xml` file (for example, the purple color resources used in the default app theme).

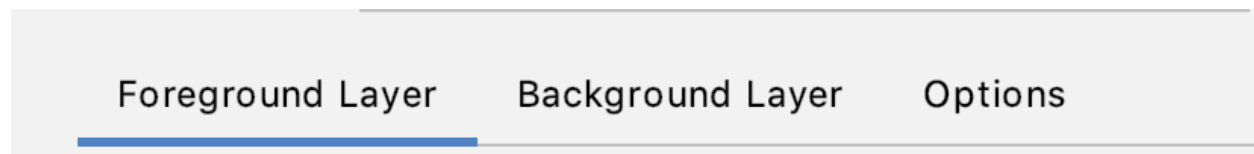
Change the app icon

As a final step, you'll update the app icon.

1. Download the app icon files [ic_launcher_foreground.xml](#) and [ic_launcher_background.xml](#). If your browser shows the file instead of downloading it, select **File > Save Page As...** to save it to your computer.
2. Within Android Studio, delete two files: `drawable/ic_launcher_background.xml` and `drawable-v24/ic_launcher_foreground.xml` files since those are for the previous app icon. You can uncheck the box **Safe delete (with usage search)**.
3. Then right click on the **res > drawable** folder and select **New > Image Asset**.



4. In the **Configure Image Asset** window make sure **Foreground layer** is selected.



5. Below that, find the **Path** label.
6. Click the folder icon inside the **Path** text box.
7. Find and open the `ic_launcher_foreground.xml` file that you downloaded on your computer.

Foreground Layer
Background Layer
Options

Layer Name:

Source Asset

Asset Type:
☒ Image
☐ Clip Art
☐ Text

Path:

8. Switch to the **Background Layer** tab.
9. Click the **Browse** icon inside the **Path** text box.
10. Find and open the `ic_launcher_background.xml` file on your computer. No other changes are necessary.
11. Click **Next**.

Asset Studio

Configure Image Asset

Icon Type: Launcher Icons (Adaptive and Legacy)
Preview: xhdpi
☒ Show safe zone
☐ Show grid

Name:

Foreground Layer
Background Layer
Options

Layer Name:

Source Asset

Asset Type:
☐ Color
☒ Image

Path:

Scaling

Trim:
☐ Yes
☒ No

Resize:
 100 %

An icon with the same name already exists and will be overwritten.

?
Cancel
Previous
Next
Finish

12. In the **Confirm Icon Path** dialog, click **Finish**. It's OK to overwrite the existing icons.

13. For best practices, you can move the new vector drawables `ic_launcher_foreground.xml` and `ic_launcher_background.xml` into a new resource directory called `drawable-anydpi-v26`. [Adaptive icons](#) were introduced in API 26, so these resources will only be used on devices running API 26 and above (for any dpi).
14. Delete the `drawable-v24` directory if there's nothing left there.
15. Run your app and notice the beautiful new app icon in the app drawer!

1:48



Affirmati...



Calendar



Camera



Chrome



Clock



Contacts



Drive



Duo



Files



Gmail



Google



Maps



Messages



Phone



Photos



Play Mov...



Play Music



Play Store



Settings



YouTube

16. As a last step, don't forget to reformat the Kotlin and XML files in the project so your code is cleaner and follows style guidelines.

Congratulations! You created an inspiring Affirmations app.

1:46

Affirmations



I am strong.



I believe in myself.



1:46

Affirmations



I am strong.



I believe in myself.



**Each day is a new opportunity
to grow and be a better version
of myself.**

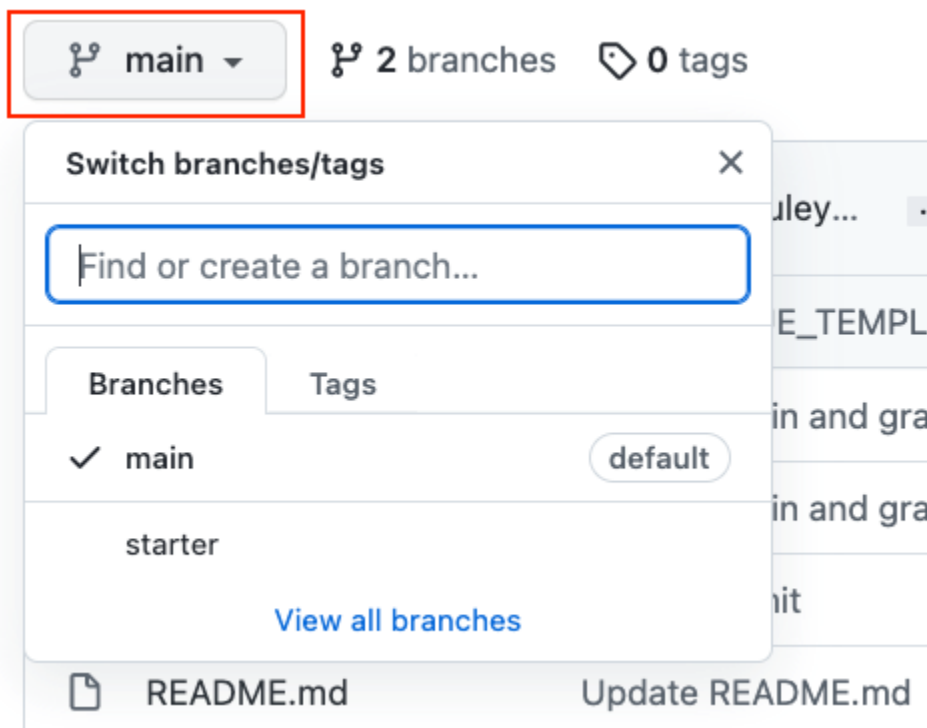
Using this knowledge of how to display a list of data in an Android app, what can you build next?

4. Solution code

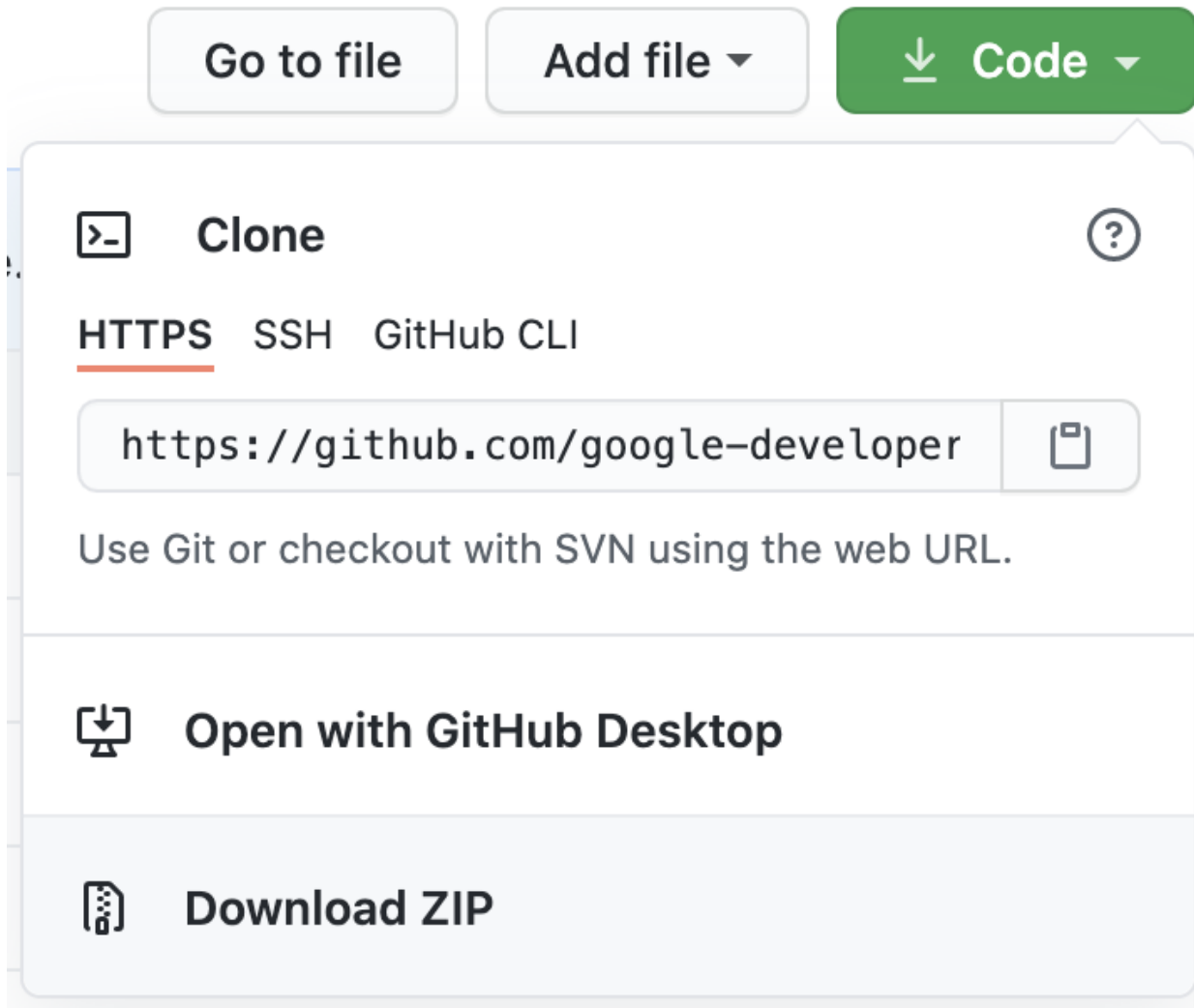
The solution code for the Affirmations app is in the GitHub repository below:

Solution Code URL: <https://github.com/google-developer-training/android-basics-kotlin-affirmations-app-solution>

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



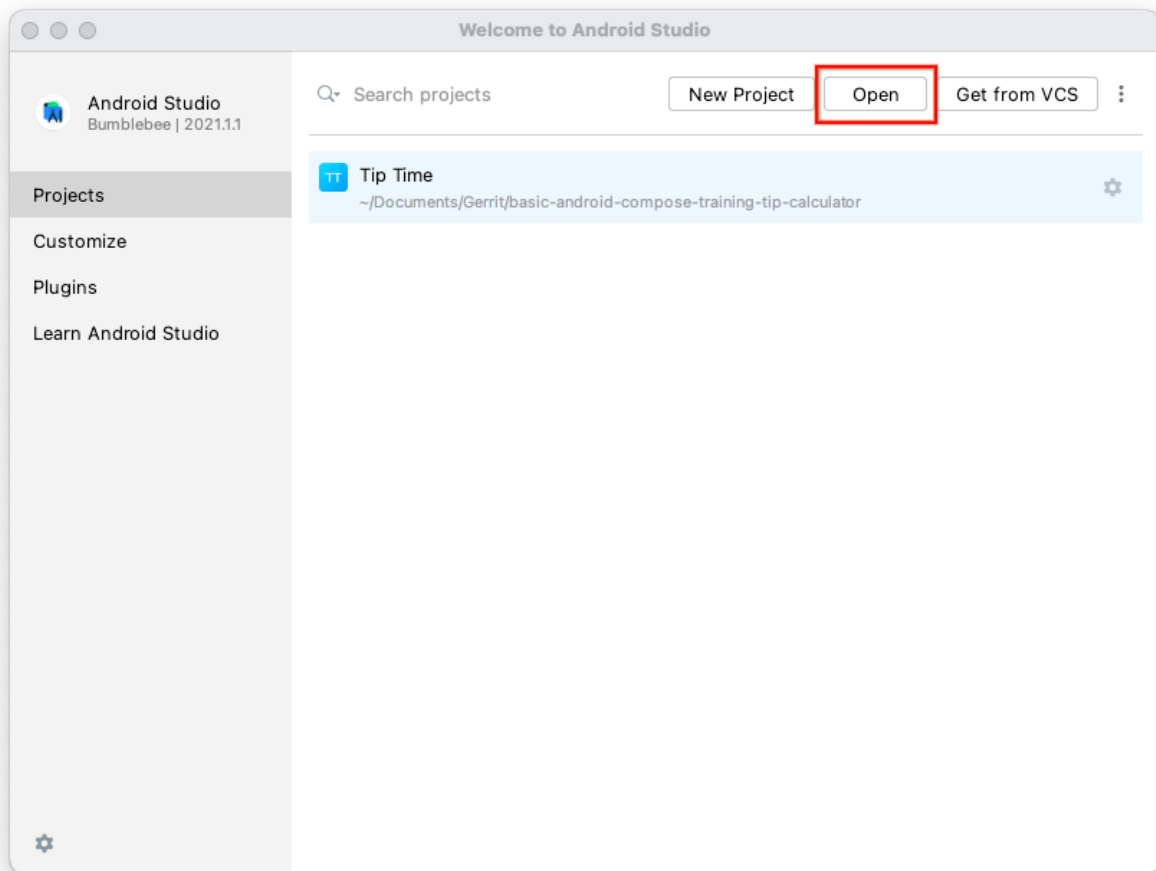
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



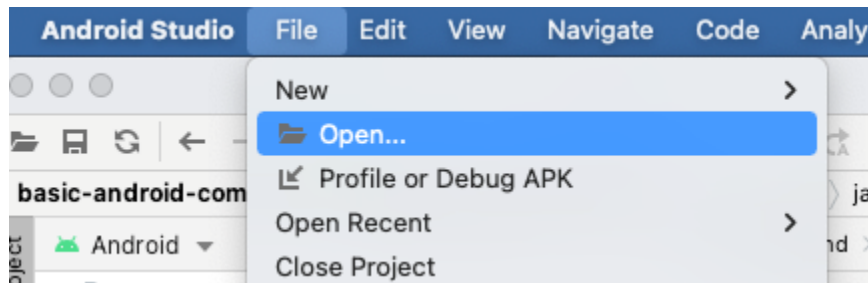
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

5. Summary

- To display additional content in a `RecyclerView`, modify the underlying data model class and data source. Then update the list item layout and adapter to set that data onto the views.
- Use resource annotations to help ensure that the right type of resource ID is passed into a class constructor.
- Use the **Material Components for Android library** to have your app more easily follow the recommended Material Design guidelines.
- Use `MaterialCardView` to display content in a Material card.
- Small visual tweaks to your app in terms of color and spacing can make the app look more polished and consistent.

6. Learn more

- [Create a list with `RecyclerView`](#)
- [RecyclerView class](#)
- [RecyclerView Adapters](#)
- [RecyclerView ViewHolder](#)
- [Lists](#) in Material Design
- [Cards](#) in Material Design
- [MaterialCardView](#)
- [Getting Started with Material Components for Android](#)
- [Android Styling: Themes vs. Styles](#)
- [Adaptive icons](#)

7. Challenge task

Note: All challenges are optional and not required for the next activity. They use and may challenge what you've learned in this and previous related codelabs.

In this series of codelabs, you've learned using `LinearLayoutManager` with `RecyclerView`. `RecyclerView` can use different `LayoutManagers` to layout data differently.

- Change the `layoutManager` property of the `RecyclerView` to `GridLayoutManager`.
- Change the column count to 3.
- Change the adapter layout to visualize data in a grid.